

Sets in Go: past, present and future

Luciano Ramalho
luciano@ramalho.org
github/codeberg:
@ramalho



About me



- Author of *Fluent Python* (O'Reilly), published in 9 languages, 2 editions (2015, 2022)
 - member of the Bartz v. Anthropic class action
- Principal consultant at Thoughtworks (2015-2023)
- Workshopping and hacking at Oficina Turing
- Co-founder of Garoa Hacker Clube, a hackerspace in São Paulo, Brasil (since 2010)

A tale of two talks

- In 2018 I presented "Set Practice" at GopherCon Brasil:
 - Pitch: "Why and how to implement sets in Go"
- This is a radical update, covering the set types in the collections package for Go 1.28

vintage
2018

ThoughtWorks®

construção e uso

PRÁTICA DE CONJUNTOS

Porquê e como implementar um tipo Set na linguagem Go.



Luciano Ramalho
@ramalhoorg

USE CASE #1

**display product if all
words in the query appear
in the product
description.**

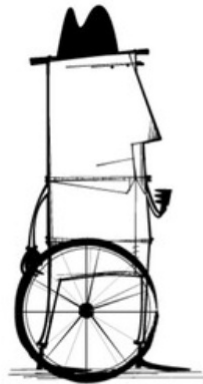
HAND-ROLLED SOLUTION #1

vintage
2018

I've written code like this in Go, which lacks built-in sets:

```
func ContainsAll(slice, subslice []string) bool {  
    for _, needle := range subslice {  
        found := false  
        for _, elem := range slice {  
            if needle == elem {  
                found = true  
                break  
            }  
        }  
        if !found {  
            return false  
        }  
    }  
    return true  
}
```


What if...



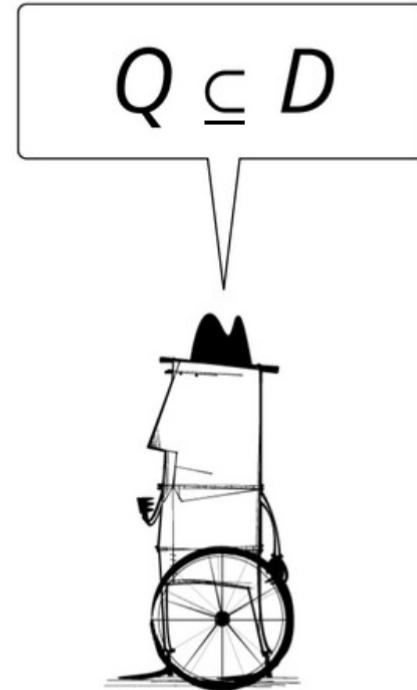
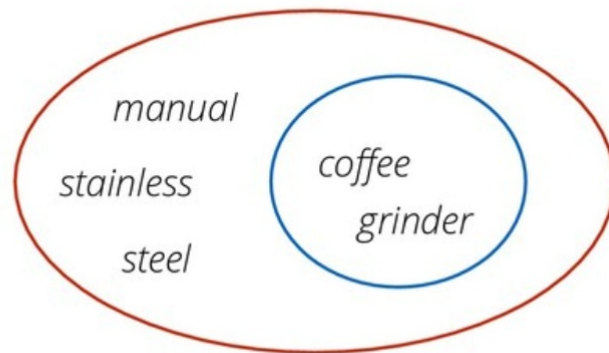
Later! I am too busy
coding nested loops!



USE CASE #1

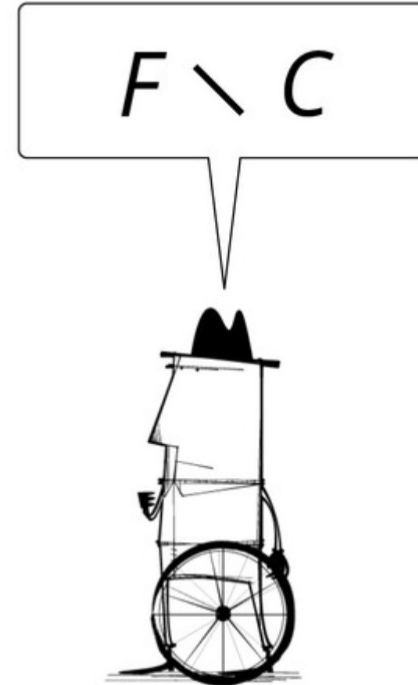
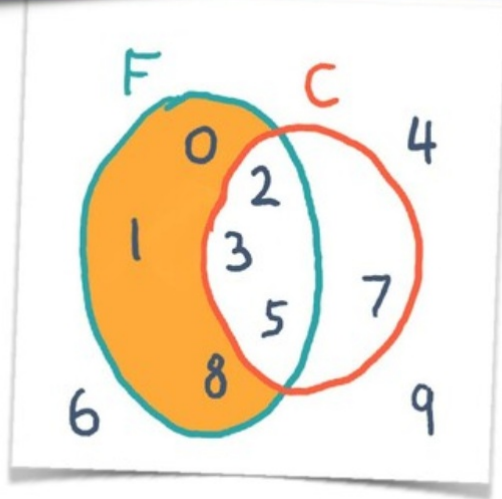
www.workcompass.com/

display product if all
words in the query appear
in the product
description.



USE CASE #2

Mark all products previously favorited, except those already in the shopping cart.



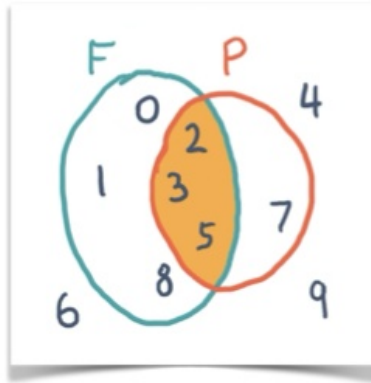


Sets and Logic

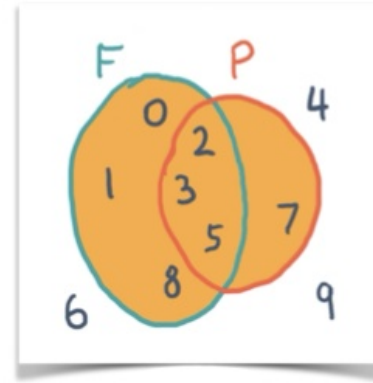
Nobody has yet discovered a branch of mathematics that has successfully resisted formalization into set theory.

Thomas Forster
Logic Induction and Sets
p. 167

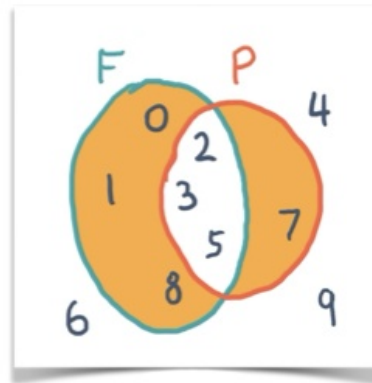
SOME SET OPERATIONS AND LOGIC EQUIVALENTS



Intersection
a.k.a. logical
conjunction



Union
a.k.a. logical
disjunction



**Symmetric
difference**
a.k.a. negation
of conjunction

LOGIC CONJUNCTION IS INTERSECTION

x belongs to the intersection of A with B.

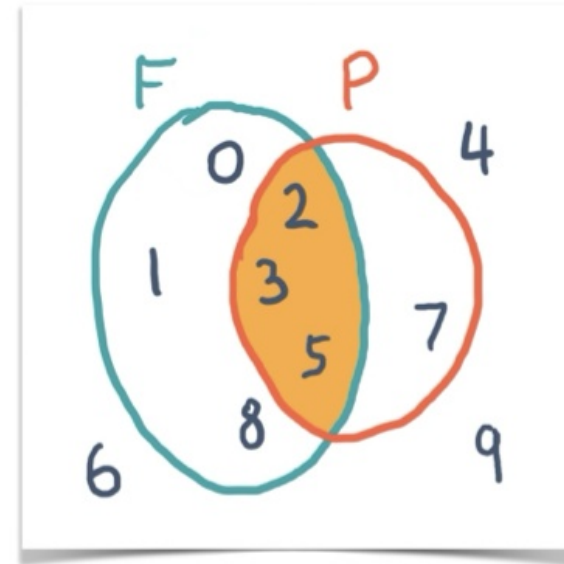
is the same as:

*x belongs to A **and**
x also belongs to B.*

Math notation:

$$x \in (A \cap B) \iff (x \in A) \wedge (x \in B)$$

In computing: **AND**



LOGIC DISJUNCTION: UNION

x belongs to the union of A and B.

is the same as:

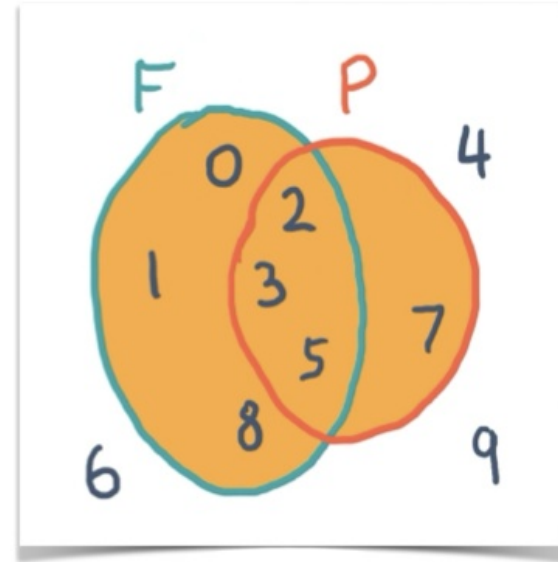
*x belongs to A **or***

x belongs to B.

Math notation:

$$x \in (A \cup B) \iff (x \in A) \vee (x \in B)$$

In computing: **OR**



DIFFERENCE

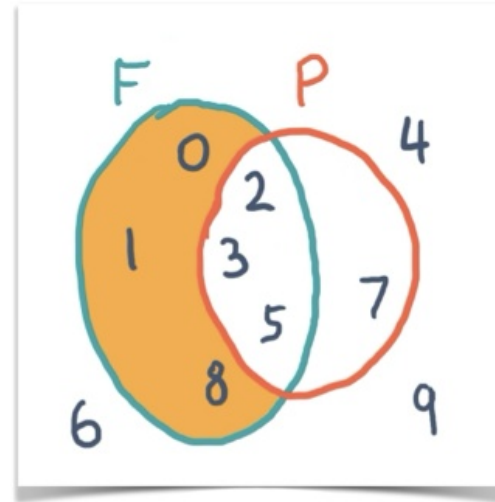
*x belongs to A but
does not belong to B .*

is the same as:

*elements of A minus
elements of B*

Math notation:

$$x \in (A \setminus B) \iff (x \in A) \wedge (x \notin B)$$



SYMMETRIC DIFFERENCE

*x belongs to A **or**
 x belongs to B but
does not belong to both*

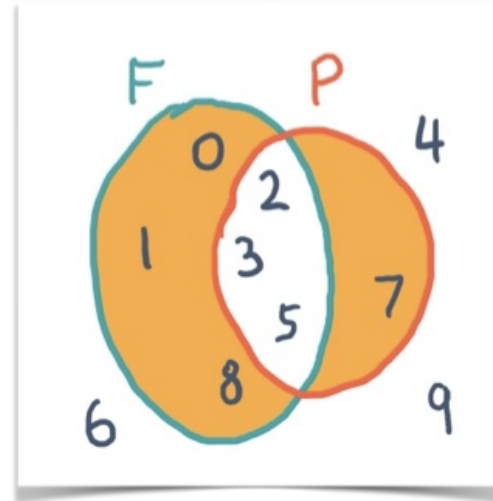
Is the same as:

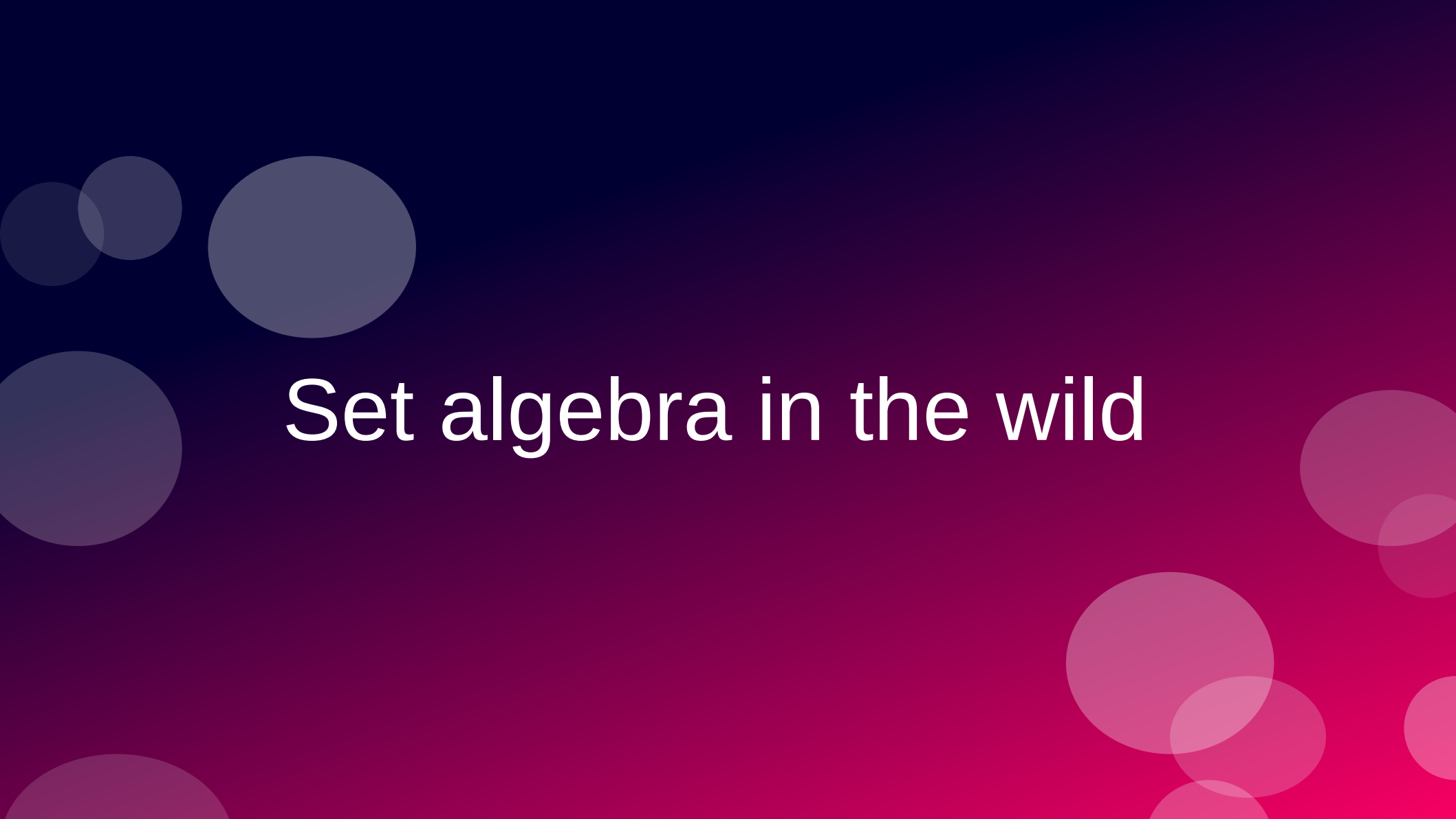
*x belongs to the union of A with B
less the intersection of A with B .*

Math notation:

$$x \in (A \Delta B) \iff (x \in A) \vee (x \in B)$$

In computing: **XOR**





Set algebra in the wild

Implementing sets

- Invariant: set elements are unique
 - Elements must be hashable and comparable
- $e \in S$ (membership test):
 - Expected to be $O(1)$ in a reasonable implementation

Set algebra

- Essential: $e \in S$
- Highly desirable:
 - $S \supseteq Z$ (superset; S contains all elements of Z)
 - $S \cap Z$ (intersection)
 - $S \cup Z$ (union)
 - $S \setminus Z$ (difference)

SETS IN A FEW LANGUAGES AND PLATFORMS

Some languages/platform APIs that implement sets in their standard libraries

Elixir	MapSet: 14 methods	😺
Ruby	Set: > 10 methods and operators	😺
Python	set, frozenset: > 10 methods and operators	😺
.Net (C# etc.)	ISet interface: > 10 methods; 2 implementations	😺
JavaScript (ES6)	Set: < 10 methods	
Java	Set interface: < 10 methods; 8 implementations	
Go	Do it yourself, or pick one of several packages...	

vintage
2018

SETS IN A FEW LANGUAGES AND PLATFORMS

Some languages/platform APIs that implement sets in their standard libraries

Elixir	MapSet: 14 methods	😺
Ruby	Set: > 10 methods and operators	😺
Python	set, frozenset: > 10 methods and operators	😺
.Net (C# etc.)	ISet interface: > 10 methods; 2 implementations	😺
JavaScript (ES6)	Set: < 10 methods	😺
Java	Set interface: < 10 methods; 8 implementations	😺
Go	Do it yourself, or pick one of several packages...	😼

vintage
2018

SETS IN A FEW LANGUAGES AND PLATFORMS

Some languages/platform APIs that implement sets in their standard libraries

Elixir	MapSet: 14 methods		} rich APIs
Ruby	Set: > 10 methods and operators		
Python	set, frozenset: > 10 methods and operators		
.Net (C# etc.)	ISet interface: > 10 methods; 2 implementations		
JavaScript (ES6)	Set: < 10 methods		} lean APIs
Java	Set interface: < 10 methods; 8 implementations		
Go	Do it yourself, or pick one of several packages...		

vintage
2018

Evolution of set APIs

- Most methods in lean APIs handle single elements
 - e.g.: adding/removing an element, iteration
- “The von Neumann bottleneck”

Can Programming Be Liberated from the von Neumann Style? A Functional Style and Its Algebra of Programs

John Backus
IBM Research Laboratory, San Jose



General permission to make fair use in teaching or research of all or part of this material is granted to individual readers and to nonprofit libraries acting for them provided that ACM's copyright notice is given and that reference is made to the publication, to its date of issue, and to the fact that reprinting privileges were granted by permission of the Association for Computing Machinery. To otherwise reprint a figure, table, other substantial excerpt, or the entire work requires specific permission as does republication, or systematic or multiple reproduction.

Author's address: 91 Saint Germain Ave., San Francisco, CA 94114.

© 1978 ACM 0001-0782/78/0800-0613 \$00.75

613

Conventional programming languages are growing ever more enormous, but not stronger. Inherent defects at the most basic level cause them to be both fat and weak: their primitive word-at-a-time style of programming inherited from their common ancestor—the von Neumann computer, their close coupling of semantics to state transitions, their division of programming into a world of expressions and a world of statements, their inability to effectively use powerful combining forms for building new programs from existing ones, and their lack of useful mathematical properties for reasoning about programs.

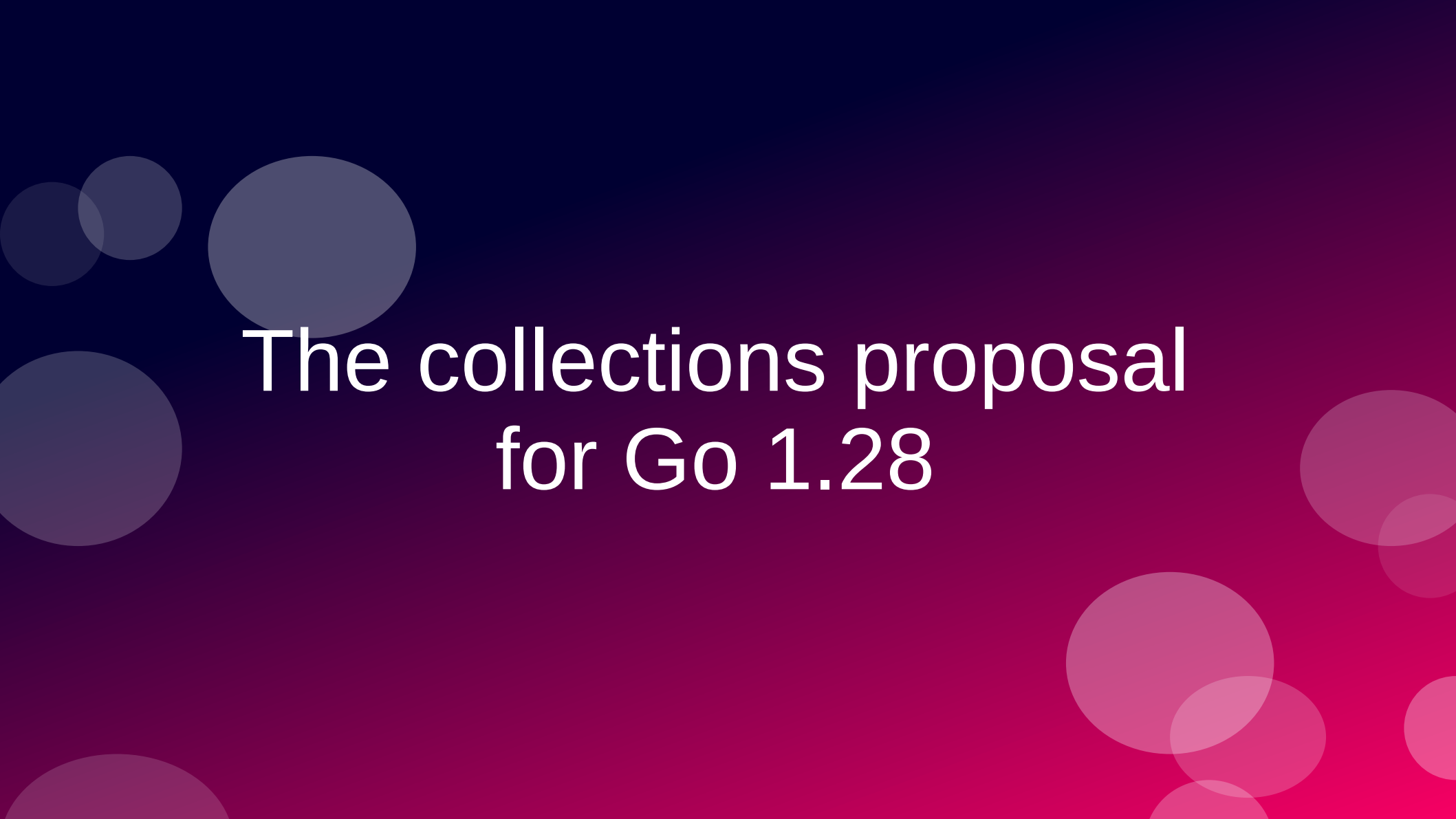
An alternative functional style of programming is founded on the use of combining forms for creating programs. Functional programs deal with structured data, are often nonrepetitive and nonrecursive, are hierarchically constructed, do not name their arguments, and do not require the complex machinery of procedure declarations to become generally applicable. Combining forms can use high level programs to build still higher level ones in a style not possible in conventional languages.

Communications
of
the ACM

August 1978
Volume 21
Number 8

Evolution of set APIs

- Languages are providing more set algebra operations
- ECMAScript 2025 added:
 - `Set.intersection`, `Set.union`,
 - `Set.difference`,
 - `Set.symmetricDifference`,
 - `Set.isSubsetOf`, `Set.isSupersetOf`,
 - `Set.isDisjointFrom`
- Go 1.28 will have a rich set API!



The collections proposal for Go 1.28

The Go Collections working group was formed in late 2025 with the purpose of bringing common collection data structures to the standard library, guided by the familiar Go principles of pragmatism and simplicity.

...

This work seeks to add several of the more important data types to the standard library, and to establish conventions for their APIs and those of future additions.

Alan Donovan
Go issue #80590

proposal: container/...: generic collection types

- Umbrella issue:
<https://github.com/golang/go/issues/80590>
- 6 new concrete collection types
- 1 new interface Hasher (included in Go 1.27)
- 3 new abstract interfaces for collections, sets and maps

Source tree (WIP)

Open CIs
(unmerged)
identified
by #issue

go/src/	
└─ hash/	
└─ maphash/	
└─ maphash.go	= Hasher interface, in Go 1.27 (#70471)
└─ maps/	
└─ maps.go	★ added Identical (#78456), used by mapset
└─ container/	
└─ container_test.go	★+ abstract interfaces (unexported)
└─ set/	
└─ set.go	★+ the new canonical set (#69230)
└─ mapset/	
└─ mapset.go	★+ helpers for map-based sets (#77052)
└─ hash/	
└─ map.go	★+ hash.Map, uses maphash.Hasher (#69559)
└─ set.go	★+ hash.Set, same approach (#80584)
└─ heap/	
└─ heap.go	= existing, pre-generics
└─ v2/	
└─ heap.go	? generic, more ergonomic Heap (#77397)
└─ list/	
└─ list.go	= existing
└─ ordered/	
└─ ordered.go	? tree-based ordered.Map/Set (#60630)
└─ ring/	
└─ ring.go	= existing

Abstract classes

go/src/

- |— hash/
 - |— |— maphash/
 - |— |— maphash.go
- |— maps/
 - |— |— maps.go
- |— container/
 - |— container/
 - |— |— container_test.go
- |— |— set/
 - |— |— |— set.go
- |— |— mapset/
 - |— |— |— mapset.go
- |— |— hash/
 - |— |— |— map.go
 - |— |— |— set.go

The canonical Set type

```
go/src/  
├── hash/  
│   └── maphash/  
│       └── maphash.go  
├── maps/  
│   └── maps.go  
├── container/  
├── container/  
│   ├── container_test.go  
│   ├── set/  
│   │   └── set.go  
│   ├── mapset/  
│   │   └── mapset.go  
│   └── hash/  
│       ├── map.go  
│       └── set.go
```

Functions for using legacy maps as sets

```
go/src/  
├── hash/  
│   └── maphash/  
│       └── maphash.go  
├── maps/  
│   └── maps.go  
├── container/  
├── container/  
│   ├── container_test.go  
│   ├── set/  
│   │   └── set.go  
│   ├── mapset/  
│   │   └── mapset.go  
│   └── hash/  
│       ├── map.go  
│       └── set.go
```

Hasher interface

```
go/src/
├── hash/
│   └── maphash/
│       └── maphash.go
├── maps/
│   └── maps.go
├── container/
├── container/
│   ├── container_test.go
│   ├── set/
│   │   └── set.go
│   ├── mapset/
│   │   └── mapset.go
│   └── hash/
│       ├── map.go
│       └── set.go
```

Set type with support for custom hashers

```
go/src/  
├── hash/  
│   └── maphash/  
│       └── maphash.go  
├── maps/  
│   └── maps.go  
├── container/  
├── container/  
│   ├── container_test.go  
│   ├── set/  
│   │   └── set.go  
│   ├── mapset/  
│   │   └── mapset.go  
│   └── hash/  
│       ├── map.go  
│       └── set.go
```

The abstract interfaces

- Defined with tests in `go/src/container/container_test.go`
- Unexported but intended as models for new collection types

Comment in container/container_test.go

```
• // The following interfaces define the abstract data types for
• // collections in Go. [...]
• // ...
• //
• // These interfaces are not yet published, but may be included in a
• // future Go release once we have experience of whether these methods
• // are necessary and sufficient.
• // ...
• // [interface definitions...]
• //
• // These types define the fundamental operations that need to be
• // implemented by all set and map types. Their naming, signature, and
• // semantic conventions should be followed wherever possible when
• // defining new collection types.
```

_AbstractCollection

```
1 // _AbstractCollection models a collection C of elements E,  
2 // such as *hash.Map, *hash.Set, *ordered.Map, or set.Set.  
3 type _AbstractCollection[E any, C _AbstractCollection[E, C]] interface {  
4     Clear()  
5     Clone() C  
6     Contains(E) bool  
7     ContainsAll(iter.Seq[E]) bool  
8     Len() int  
9     String() string  
10 }
```

_AbstractSet

```
1 // _AbstractSet models a set S of elements E,  
2 // such as *hash.Set, or set.Set.  
3 type _AbstractSet[E any, S _AbstractSet[E, S]] interface {  
4     _AbstractCollection[E, S]  
5  
6     Insert(E) bool  
7     InsertAll(iter.Seq[E]) bool  
8     Equal(S) bool  
9     All() iter.Seq[E]  
10    Delete(E) bool  
11    DeleteAll(iter.Seq[E]) bool  
12    DeleteFunc(func(E) bool) bool  
13    Intersection(S) S  
14    IntersectionWith(S)  
15    Intersects(S) bool  
16    Union(S) S  
17    UnionWith(S)  
18    Difference(S) S  
19    DifferenceWith(S)  
20    SymmetricDifference(S) S  
21    SymmetricDifferenceWith(S)  
22 }
```



```
1 // _AbstractSet models a set S of elements E,  
2 // such as *hash.Set, or set.Set.  
3 type _AbstractSet[E any, S _AbstractSet[E, S]] interface {  
4     _AbstractCollection[E, S]  
5  
6     Insert(E) bool  
7     InsertAll(iter.Seq[E]) bool  
8     Equal(S) bool  
9     All() iter.Seq[E]  
10    Delete(E) bool  
11    DeleteAll(iter.Seq[E]) bool  
12    DeleteFunc(func(E) bool) bool  
13    Intersection(S) S  
14    IntersectionWith(S)  
15    Intersects(S) bool  
16    Union(S) S  
17    UnionWith(S)  
18    Difference(S) S  
19    DifferenceWith(S)  
20    SymmetricDifference(S) S  
21    SymmetricDifferenceWith(S)  
22 }
```



set algebra

_AbstractMap

```
1 // _AbstractMap models a mapping M from keys K to values V,  
2 // such as *hash.Map or *ordered.Map.  
3 type _AbstractMap[K, V any, M _AbstractMap[K, V, M]] interface {  
4     _AbstractCollection[K, M]  
5  
6     Set(K, V) (V, bool)  
7     SetAll(iter.Seq2[K, V]) bool  
8     Get(K) (V, bool)  
9     At(K) V  
10    All() iter.Seq2[K, V]  
11    Keys() iter.Seq[K]  
12    Values() iter.Seq[V]  
13    Delete(K) (V, bool)  
14    DeleteAll(iter.Seq[K]) bool  
15    DeleteFunc(func(K, V) bool) bool  
16 }
```

The 7 proposed additions

1) `hash/maphash.Hasher`

standard interface supporting custom hash functions and equivalence relations for arbitrary data types (released in Go 1.27)

2) `container/hash.Map[K, V]`

a hash-based Map that uses custom hash functions.

3) `container/hash.Set[T]`

a hash-based Set along the same lines.

4) `container/heap/v2.Heap`

a generic binary heap API to replace the standard library's existing heap (hard to use).

The 7 proposed additions

5) `container/set.Set[T]`

transparently represented as `map[T]struct{}`, supporting usual set operations such as Union and Intersection. We expect it to become the standard set in most new Go APIs.

6) `container/mapset`

helper functions (Union, Intersection, and so on) for manipulating legacy sets as sets in existing code whose API cannot be changed. `set.Set` wraps this API.

7) `container/ordered.Map[K, V]`

a map that preserves insertion order

The new canonical set type

```
1 package set
2
3 import (
4     "github.com/ramalho/sets-in-go/vendored/mapset" // proposed mapset
5     "iter"
6     "maps"
7 )
8
9 // A Set[E] is a set of elements of type E.
10 type Set[E comparable] map[E]struct{}
11
12 // Collect creates a new set containing the elements of the sequence.
13 func Collect[E comparable](seq iter.Seq[E]) Set[E] {
14     return Set[E](mapset.Collect(seq))
15 }
```

The underlying mapset (uses no named types)

```
1 package mapset
2
3 import (
4     "iter"
5 )
6
7 // Collect returns a new set containing the elements of the sequence.
8 func Collect[K comparable](seq iter.Seq[K]) map[K]struct{} {
9     return collect[K, struct{}](seq)
10 }
11
12 /// ...
13
14 func collect[K comparable, V bool | struct{}](seq iter.Seq[K]) map[K]V {
15     x := make(map[K]V)
16     InsertAll(x, seq)
17     return x
18 }
```

mapset.InsertAll and mapset.insert

```
1 // InsertAll adds each element of the addenda sequence to the set.
2 // If the set values are boolean, the value 'true' is used.
3 // It reports whether len(x) changed.
4 func InsertAll[M ~map[K]V, K comparable, V bool | struct{}](x M, addenda iter.Seq[K]) bool {
5     pre := len(x)
6     for k := range addenda {
7         insert(x, k)
8     }
9     return len(x) != pre
10 }
11
12 func insert[M ~map[K]V, K comparable, V bool | struct{}](m M, k K) {
13     // Choose the distinguished "present" value (true or struct{}{}).
14     // This compiles to a load from .rodata.
15     var present V
16     if _, ok := any(present).(bool); ok {
17         present = any(true).(V)
18     }
19
20     // This is the canonical insertion operation.
21     // All maps created by this API use only the
22     // distinguished 'present' value for the result type.
23     m[k] = present
24 }
```



Wrapping up

Key takeaways

- Set operations provide simpler, faster solutions to many coding tasks.
- Sets and other collections in the upcoming Go 1.28 are great examples of pragmatic design for generic collections.
- The precise vocabulary of set algebra is useful to prompt coding agents, regardless of the programming language.